CHAPTER 5

Identifying Architectural Characteristics

Identifying the driving architectural characteristics is one of the first steps in creating an architecture or determining the validity of an existing architecture. Identifying the correct architectural characteristics ("-ilities") for a given problem or application requires an architect to not only understand the domain problem, but also collaborate with the problem domain stakeholders to determine what is truly important from a domain perspective.

An architect uncovers architecture characteristics in at least three ways by extracting from domain concerns, requirements, and implicit domain knowledge. We previously discussed implicit characteristics and we cover the other two here.

Extracting Architecture Characteristics from Domain Concerns

An architect must be able to translate domain concerns to identify the right architectural characteristics. For example, is scalability the most important concern, or is it fault tolerance, security, or performance? Perhaps the system requires all four characteristics combined. Understanding the key domain goals and domain situation allows an architect to translate those domain concerns to "-ilities," which then forms the basis for correct and justifiable architecture decisions.

One tip when collaborating with domain stakeholders to define the driving architecture characteristics is to work hard to keep the final list as short as possible. A common anti-pattern in architecture entails trying to design a *generic architecture*, one that supports *all* the architecture characteristics. Each architecture characteristic the architecture supports complicates the overall system design; supporting too many architecture characteristics leads to greater and greater complexity before the architect and developers have even started addressing the problem domain, the original

motivation for writing the software. Don't obsess over the number of charateristics, but rather the motivation to keep design simple.

Case Study: The Vasa

The original story of over-specifying architecture characteristics and ultimately killing a project must be the Vasa. It was a Swedish warship built between 1626 and 1628 by a king who wanted the most magnificent ship ever created. Up until that time, ships were either troop transports or gunships—the Vasa would be both! Most ships had one deck—the Vasa had two! All the cannons were twice the size of those on similar ships. Despite some trepidation by the expert ship builders (who ultimately couldn't say no to King Adolphus), the shipbuilders finished the construction. In celebration, the ship sailed out into the harbor and shot a cannon salute off one side. Unfortunately, because the ship was top-heavy, it capsized and sank to the bottom of the bay in Sweden. In the early 20th century, salvagers rescued the ship, which now resides in a museum in Stockholm.

Many architects and domain stakeholders want to prioritize the final list of architecture characteristics that the application or system must support. While this is certainly desirable, in most cases it is a fool's errand and will not only waste time, but also produce a lot of unnecessary frustration and disagreement with the key stakeholders. Rarely will all stakeholders agree on the priority of each and every characteristic. A better approach is to have the domain stakeholders select the top three most important characteristics from the final list (in any order). Not only is this much easier to gain consensus on, but it also fosters discussions about what is most important and helps the architect analyze trade-offs when making vital architecture decisions.

Most architecture characteristics come from listening to key domain stakeholders and collaborating with them to determine what is important from a domain perspective. While this may seem like a straightforward activity, the problem is that architects and domain stakeholders speak different languages. Architects talk about scalability, interoperability, fault tolerance, learnability, and availability. Domain stakeholders talk about mergers and acquisitions, user satisfaction, time to market, and competitive advantage. What happens is a "lost in translation" problem where the architect and domain stakeholder don't understand each other. Architects have no idea how to create an architecture to support user satisfaction, and domain stakeholders don't understand why there is so much focus and talk about availability, interoperability, learnability, and fault tolerance in the application. Fortunately, there is usually a translation from domain concerns to architecture characteristics. Table 5-1 shows some of the more common domain concerns and the corresponding "-ilities" that support them.

Table 5-1. Translation of domain concerns to architecture characteristics

Domain concern	Architecture characteristics
Mergers and acquisitions	Interoperability, scalability, adaptability, extensibility
Time to market	Agility, testability, deployability
User satisfaction	Performance, availability, fault tolerance, testability, deployability, agility, security
Competitive advantage	Agility, testability, deployability, scalability, availability, fault tolerance
Time and budget	Simplicity, feasibility

One important thing to note is that agility does not equal time to market. Rather, it is agility + testability + deployability. This is a trap many architects fall into when translating domain concerns. Focusing on only one of the ingredients is like forgetting to put the flour in the cake batter. For example, a domain stakeholder might say something like "Due to regulatory requirements, it is absolutely imperative that we complete end-of-day fund pricing on time." An ineffective architect might just focus on performance because that seems to be the primary focus of that domain concern. However, that architect will fail for many reasons. First, it doesn't matter how fast the system is if it isn't available when needed. Second, as the domain grows and more funds are created, the system must be able to also scale to finish end-of-day processing in time. Third, the system must not only be available, but must also be reliable so that it doesn't crash as end-of-day fund prices are being calculated. Forth, what happens if the end-of-day fund pricing is about 85% complete and the system crashes? It must be able to recover and restart where the pricing left off. Finally, the system may be fast, but are the fund prices being calculated correctly? So, in addition to performance, the architect must also equally place a focus on availability, scalability, reliability, recoverability, and auditability.

Extracting Architecture Characteristics from Requirements

Some architecture characteristics come from explicit statements in requirements documents. For example, explicit expected numbers of users and scale commonly appear in domain or domain concerns. Others come from inherent domain knowledge by architects, one of the many reasons that domain knowledge is always beneficial for architects. For example, suppose an architect designs an application that handles class registration for university students. To make the math easy, assume that the school has 1,000 students and 10 hours for registration. Should an architect design a system assuming consistent scale, making the implicit assumption that the students during the registration process will distribute themselves evenly over time? Or, based on knowledge of university students habits and proclivities, should the architect design a system that can handle all 1,000 students attempting to register in the last 10 minutes? Anyone who understands how much students stereotypically

procrastinate knows the answer to this question! Rarely will details like this appear in requirements documents, yet they do inform the design decisions.

The Origin of Architecture Katas

A few years ago, Ted Neward, a well-known architect, devised architecture katas, a clever method to allow nascent architects a way to practice deriving architecture characteristics from domain-targeted descriptions. From Japan and martial arts, a *kata* is an individual training exercise, where the emphasis lies on proper form and technique.

> How do we get great designers? Great designers design, of course.
>
> —Fred Brooks

So how are we supposed to get great architects if they only get the chance to architect fewer than a half dozen times in their career?

To provide a curriculum for aspiring architects, Ted created the first architecture katas site, which your authors Neal and Mark adapted and updated. The basic premise of the kata exercise provides architects with a problem stated in domain terms and additional context (things that might not appear in requirements yet impact design). Small teams work for 45 minutes on a design, then show results to the other groups, who vote on who came up with the best architecture. True to its original purpose, architecture katas provide a useful laboratory for aspiring architects.

Each kata has predefined sections:

Description
: The overall domain problem the system is trying to solve

Users
: The expected number and/or types of users of the system

Requirements
: Domain/domain-level requirements, as an architect might expect from domain users/domain experts

Neal updated the format a few years later on his blog to add the *additional context* section to each kata with important additional considerations, making the exercises more realistic.

Additional context
: Many of the considerations an architect must make aren't explicitly expressed in requirements but rather by implicit knowledge of the problem domain

We encourage burgeoning architects to use the site to do their own kata exercise. Anyone can host a brown-bag lunch where a team of aspiring architects can solve a problem and get an experienced architect to evaluate the design and trade-off

analysis, either on the spot or from a short analysis after the fact. The design won't be elaborate because the exercise is timeboxed. Team members ideally get feedback from the experienced architecture about missed trade-offs and alternative designs.

Case Study: Silicon Sandwiches

To illustrate several concepts, we use an *architecture kata* (see "The Origin of Architecture Katas" on page 68 for the origin of the concept). To show how architects derive architecture characteristics from requirements, we introduce the Silicon Sandwiches kata.

Description

A national sandwich shop wants to enable online ordering (in addition to its current call-in service).

Users

Thousands, perhaps one day millions

Requirements

- Users will place their order, then be given a time to pick up their sandwich and directions to the shop (which must integrate with several external mapping services that include traffic information)
- If the shop offers a delivery service, dispatch the driver with the sandwich to the user
- Mobile-device accessibility
- Offer national daily promotions/specials
- Offer local daily promotions/specials
- Accept payment online, in person, or upon delivery

Additional context

- Sandwich shops are franchised, each with a different owner
- Parent company has near-future plans to expand overseas
- Corporate goal is to hire inexpensive labor to maximize profit

Given this scenario, how would an architect derive architecture characteristics? Each part of the requirement might contribute to one or more aspects of architecture (and many will not). The architect doesn't design the entire system here—considerable effort must still go into crafting code to solve the domain statement. Instead, the architect looks for things that influence or impact the design, particularly structural.

First, separate the candidate architecture characteristics into explicit and implicit characteristics.

Explicit Characteristics

Explicit architecture characteristics appear in a requirements specification as part of the necessary design. For example, a shopping website may aspire to support a particular number of concurrent users, which domain analysts specify in the requirements. An architect should consider each part of the requirements to see if it contributes to an architecture characteristic. But first, an architect should consider domain-level predictions about expected metrics, as represented in the Users section of the kata.

One of the first details that should catch an architect's eye is the number of users: currently thousands, perhaps one day millions (this is a very ambitious sandwich shop!). Thus, *scalability*—the ability to handle a large number of concurrent users without serious performance degradation—is one of the top architecture characteristics. Notice that the problem statement didn't explicitly ask for scalability, but rather expressed that requirement as an expected number of users. Architects must often decode domain language into engineering equivalents.

However, we also probably need *elasticity*—the ability to handle bursts of requests. These two characteristics often appear lumped together, but they have different constraints. Scalability looks like the graph shown in Figure 5-1.

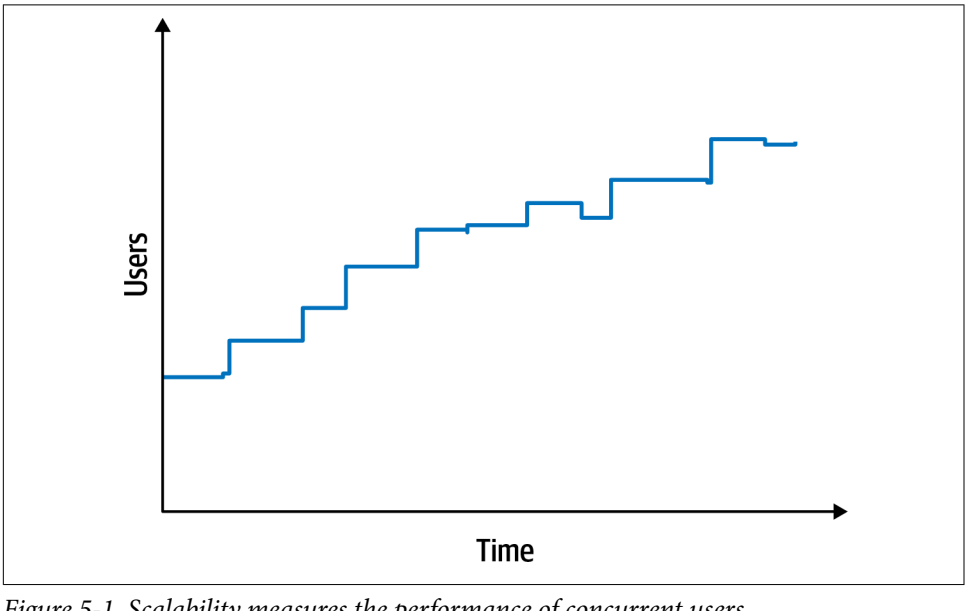

Figure 5-1. Scalability measures the performance of concurrent users

Elasticity, on the other hand, measures bursts of traffic, as shown in Figure 5-2.

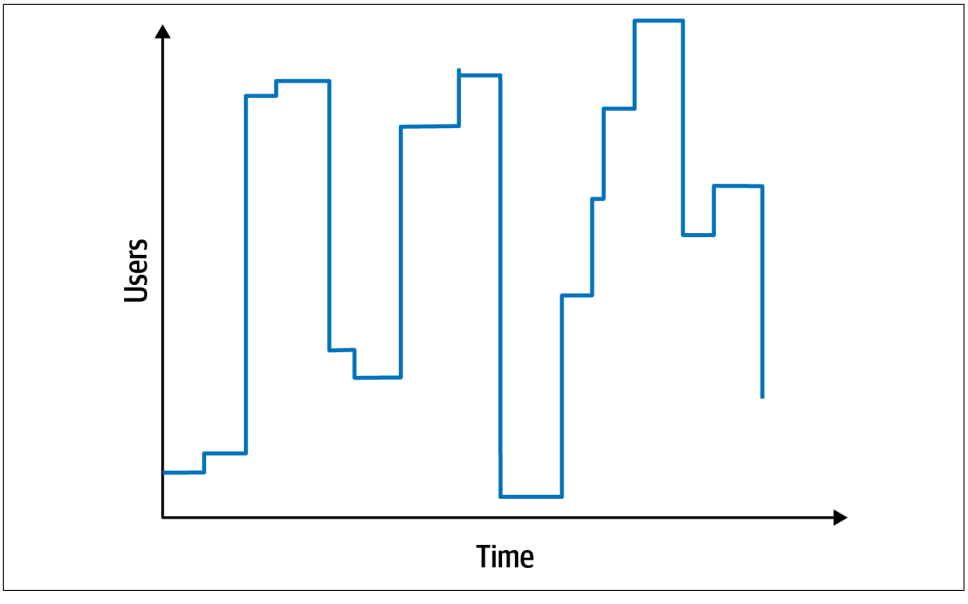

Figure 5-2. Elastic systems must withstand bursts of users

Some systems are scalable but not elastic. For example, consider a hotel reservation system. Absent special sales or events, the number of users is probably consistent. In contrast, consider a concert ticket booking system. As new tickets go on sale, fervent fans will flood the site, requiring high degrees of elasticity. Often, elastic systems also need scalability: the ability to handle bursts and high numbers of concurrent users.

The requirement for elasticity did not appear in the Silicon Sandwiches requirements, yet the architect should identify this as an important consideration. Requirements sometimes state architecture characteristics outright, but some lurk inside the problem domain. Consider a sandwich shop. Is its traffic consistent throughout the day? Or does it endure bursts of traffic around mealtimes? Almost certainly the latter. Thus, a good architect should identify this potential architecture characteristic.

An architect should consider each of these business requirements in turn to see if architecture characteristics exist:

1. Users will place their order, then be given a time to pick up their sandwich and directions to the shop (which must provide the option to integrate with external mapping services that include traffic information).

   External mapping services imply integration points, which may impact aspects such as reliability. For example, if a developer builds a system that relies on a third-party system, yet calling it fails, it impacts the reliability of the calling system. However, architects must also be wary of over-specifying architecture characteristics. What if the external traffic service is down? Should the Silicon

Sandwiches site fail, or should it just offer slightly less efficiency without traffic information? Architects should always guard against building unnecessary brittleness or fragility into designs.

2. If the shop offers a delivery service, dispatch the driver with the sandwich to the user.

   No special architecture characteristics seem necessary to support this requirement.

3. Mobile-device accessibility.

   This requirement will primarily affect the *design* of the application, pointing toward building either a portable web application or several native web applications. Given the budget constraints and simplicity of the application, an architect would likely deem it overkill to build multiple applications, so the design points toward a mobile-optimized web application. Thus, the architect may want to define some specific performance architecture characteristics for page load time and other mobile-sensitive characteristics. Notice that the architect shouldn't act alone in situations like this, but should instead collaborate with user experience designers, domain stakeholders, and other interested parties to vet decisions like this.

4. Offer national daily promotions/specials.
5. Offer local daily promotions/specials.

   Both of these requirements specify customizability across both promotions and specials. Notice that requirement 1 also implies customized traffic information based on address. Based on all three of these requirements, the architect may consider customizability as an architecture characteristic. For example, an architecture style such as microkernel architecture supports customized behavior extremely well by defining a plug-in architecture. In this case, the default behavior appears in the core, and developers write the optional customized parts, based on location, via plug-ins. However, a traditional design can also accommodate this requirement via design patterns (such as Template Method). This conundrum is common in architecture and requires architects to constantly weight trade-offs between competing options. We discuss particular trade-off in more detail in "Design Versus Architecture and Trade-Offs" on page 74.

6. Accept payment online, in person, or upon delivery.

   Online payments imply security, but nothing in this requirement suggests a particularly heightened level of security beyond what's implicit.

7. Sandwich shops are franchised, each with a different owner.

   This requirement may impose cost restrictions on the architecture—the architect should check the feasibility (applying constraints like cost, time, and staff skill set) to see if a simple or sacrificial architecture is warranted.

8. Parent company has near-future plans to expand overseas.

   This requirement implies *internationalization*, or *i18n*. Many design techniques exist to handle this requirement, which shouldn't require special structure to accommodate. This will, however, certainly drive design decisions.

9. Corporate goal is to hire inexpensive labor to maximize profit.

   This requirement suggests that usability will be important, but again is more concerned with design than architecture characteristics.

The third architecture characteristic we derive from the preceding requirements is *performance*: no one wants to buy from a sandwich shop that has poor performance, especially at peak times. However, *performance* is a nuanced concept—what *kind* of performance should the architect design for? We cover the various nuances of performance in Chapter 6.

We also want to define performance numbers in conjunction with scalability numbers. In other words, we must establish a baseline of performance without particular scale, as well as determine what an acceptable level of performance is given a certain number of users. Quite often, architecture characteristics interact with one another, forcing architects to define them in relation to one another.

Implicit Characteristics

Many architecture characteristics aren't specified in requirements documents, yet they make up an important aspect of the design. One implicit architecture characteristic the system might want to support is *availability*: making sure users can access the sandwich site. Closely related to availability is *reliability*: making sure the site stays up during interactions—no one wants to purchase from a site that continues dropping connections, forcing them to log in again.

Security appears as an implicit characteristic in every system: no one wants to create insecure software. However, it may be prioritized depending on criticality, which illustrates the interlocking nature of our definition. An architect considers security an architecture characteristic if it influences some structural aspect of the design and is critical or important to the application.

For Silicon Sandwiches, an architect might assume that payments should be handled by a third party. Thus, as long as developers follow general security hygiene (not passing credit card numbers as plain text, not storing too much information, and so on), the architect shouldn't need any special structural design to accommodate security; good design in the application will suffice. Each architecture characteristic interacts with the others, leading to the common pitfall of architects of over-specifying architecture characteristics, which is just as damaging as under-specifying them because it overcomplicates the system design.

The last major architecture characteristic that Silicon Sandwiches needs to support encompasses several details from the requirements: *customizability*. Notice that several parts of the problem domain offer custom behavior: recipes, local sales, and directions that may be locally overridden. Thus, the architecture should support the ability to facilitate custom behavior. Normally, this would fall into the design of the application. However, as our definition specifies, a part of the problem domain that relies on custom structure to support it moves into the realm of an architecture characteristic. This design element isn't critical to the success of the application though. It is important to note that there are no correct answers in choosing architecture characteristics, only incorrect ones (or, as Mark notes in one of his well-known quotes):

> There are no wrong answers in architecture, only expensive ones.

Design Versus Architecture and Trade-Offs

In the Silicon Sandwiches kata, an architect would likely identify customizability as a part of the system, but the question then becomes: architecture or design? The architecture implies some structural component, whereas design resides within the architecture. In the customizability case of Silicon Sandwiches, the architect could choose an architecture style like microkernel and build structural support for customization. However, if the architect chose another style because of competing concerns, developers could implement the customization using the Template Method design pattern, which allows parent classes to define workflow that can be overridden in child classes. Which design is better?

Like in all architecture, it depends on a number of factors. First, are there good reasons, such as performance and coupling, not to implement a microkernel architecture? Second, are other desirable architecture characteristics more difficult in one design versus the other? Third, how much would it cost to support all the architecture characteristics in each design versus pattern? This type of architectural trade-off analysis makes up an important part of an architect's role.

Above all, it is critical for the architect to collaborate with the developers, project manager, operations team, and other co-constructors of the software system. No architecture decision should be made isolated from the implementation team (which leads to the dreaded *Ivory Tower Architect* anti-pattern). In the case of Silicon Sandwiches, the architect, tech lead, developers, and domain analysts should collaborate to decide how best to implement customizability.

An architect could design an architecture that doesn't accommodate customizability structurally, requiring the design of the application itself to support that behavior (see "Design Versus Architecture and Trade-Offs" on page 74). Architects shouldn't stress too much about discovering the exactly correct set of architecture characteristics—developers can implement functionality in a variety of ways. However, correctly iden-

tifying important structural elements may facilitate a simpler or more elegant design. Architects must remember: there is no best design in architecture, only a least worst collection of trade-offs.

Architects must also prioritize these architecture characteristics toward trying to find the simplest required sets. A useful exercise once the team has made a first pass at identifying the architecture characteristics is to try to determine the least important one—if you must eliminate one, which would it be? Generally, architects are more likely to cull the explicit architecture characteristics, as many of the implicit ones support general success. The way we define what's critical or important to success assists architects in determining if the application truly requires each architecture characteristic. By attempting to determine the least applicable one, architects can help determine critical necessity. In the case of Silicon Sandwiches, which architecture characteristic that we have identified is least important? Again, no absolute correct answer exists. However, in this case, the solution could lose either customizability or performance. We could eliminate customizability as an architecture characteristic and plan to implement that behavior as part of application design. Of the operational architecture characteristics, performance is likely the least critical for success. Of course, the developers don't mean to build an application that has terrible performance, but rather one that doesn't prioritize performance over other characteristics, such as scalability or availability.